Definition

For real-valued nets f and g,

$\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$ iff $f(x) \in O(g(x))$ and $|g(x)|$ is an asymptotic upper bound of $|f(x)|$.

$\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M|g(x)|$ iff $f(x) \in \Omega(g(x))$ and $|g(x)|$ is an asymptotic lower bound of $|f(x)|$.

$\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$ iff $f(x) \in \Theta(g(x))$.

$\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M|g(x)|$ iff $f(x) \in o(g(x))$.

$\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M|g(x)|$ iff $f(x) \in \omega(g(x))$.

$\in$ is also denoted as $=$ for them.

Limit

If $\exists k \in \mathbb{R} : x \geq k \implies g(x) \neq 0$,

$\operatorname{limsup}\left|\frac{f(x)}{g(x)}\right| < \infty$ iff $f(x) \in O(g(x))$.

Proof. $|f(x)| \leq M|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \leq M, \quad x \geq T; \sup\left|\frac{f(x)}{g(x)}\right| \leq M, \quad x \geq T; \operatorname{limsup}\left|\frac{f(x)}{g(x)}\right| \leq M < \infty;$ □

$\operatorname{liminf}\left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Omega(g(x))$.

Proof. $|f(x)| \geq M|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \geq M, \quad x \geq T; \inf\left|\frac{f(x)}{g(x)}\right| \geq M, \quad x \geq T; \operatorname{liminf}\left|\frac{f(x)}{g(x)}\right| \geq M > 0;$ □

$\infty > \operatorname{limsup}\left|\frac{f(x)}{g(x)}\right| \geq \operatorname{liminf}\left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Theta(g(x))$.

Proof. $|f(x)| \geq M_1|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \geq M_1, \quad x \geq T; \inf\left|\frac{f(x)}{g(x)}\right| \geq M_1, \quad x \geq T; \operatorname{liminf}\left|\frac{f(x)}{g(x)}\right| \geq M_1 > 0; |f(x)| \leq M_2|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \leq M_2, \quad x \geq T; \sup\left|\frac{f(x)}{g(x)}\right| \leq M_2, \quad x \geq T; \operatorname{limsup}\left|\frac{f(x)}{g(x)}\right| \leq M_2 < \infty;$ □

$\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = 0$ iff $f(x) \in o(g(x))$. $|f(x)| < M|g(x)|; \left|\frac{f(x)}{g(x)}\right| < M; \lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = 0;$

$\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = \infty$ iff $f(x) \in \omega(g(x))$. $|f(x)| > M|g(x)|; \left|\frac{f(x)}{g(x)}\right| > M; \lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = \infty;$

Relation

For any fixed $g(x)$, $o(g(x))$, $\Theta(g(x))$, and $\omega(g(x))$ are mutually exclusive.

Proof. Prove by contradiction. If $f \in o(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$; then set $M = M_1$, for $x \geq \max\{T, U\}$ $M_1|g(x)| \leq |f(x)| < M_1|g(x)|$.; which can't happen. If $f \in \omega(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$; then set $M = M_2$, for $x \geq \max\{T, U\}$ $M_2|g(x)| \geq |f(x)| > M_2|g(x)|$.; which can't happen. If $f \in o(g(x)) \cap \omega(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M|g(x)|$; $\forall M > 0 : \exists U \in \mathbb{R} : x \geq U \implies |f(x)| > M|g(x)|$; then set any fixed M, $M|g(x)| < |f(x)| < M|g(x)|$.; which can't happen. □

For any fixed $g(x)$, $o(g(x)) \subseteq O(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\omega(g(x)) \subseteq \Omega(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\Theta(g(x)) = O(g(x)) \cap \Omega(g(x))$;

Proof. Prove $\Theta(g(x)) \subseteq O(g(x)) \cap \Omega(g(x))$; $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |f(x)| \geq N|g(x)|$; then $\exists M, N > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies N|g(x)| \leq |f(x)| \leq M|g(x)|$;

Prove $\Theta(g(x)) \supseteq O(g(x)) \cap \Omega(g(x))$; $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$; then $\exists M_2 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M_2|g(x)|$; $\exists M_1 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M_1|g(x)|$; □

For any fixed $g(x)$, $O(g(x)) \setminus (o(g(x)) \cup \Theta(g(x))) \neq \varnothing$;

Proof. Counterexample: $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $g(x) = 1$; $f \in O(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \leq |g(x)|$; $f \notin o(g(x))$: $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_1 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_1|g(x)| = M_1 > |f(x)| = 0$; □

For any fixed $g(x)$, $\Omega(g(x)) \setminus (\omega(g(x)) \cup \Theta(g(x))) \neq \varnothing$;

Proof. Counterexample: $f(x) = 1$; $g(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $f \in \Omega(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \geq |g(x)|$; $f \notin \omega(g(x))$: $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_2 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_2|g(x)| = 0 < |f(x)| = 1$; □

$f(x) \in O(g(x)) \iff g(x) \in \Omega(f(x))$;

Proof. $\implies$: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \geq \frac{1}{M}|g(x)|$; $\Leftarrow$: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M}|g(x)|$; □

$f(x) \in o(g(x)) \iff g(x) \in \omega(f(x))$;

Proof. $\implies$: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M|g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| > \frac{1}{M}|f(x)|$; $\Leftarrow$: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M|g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| < \frac{1}{M}|f(x)|$; □

For any fixed $g(x)$, $\exists f(x) : f(x) \notin O(g(x)) \cup \Omega(g(x))$;

Proof. $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ x, & x \notin \mathbb{Q} \end{cases}$; $g(x) = \begin{cases} x, & x \in \mathbb{Q} \\ 1, & x \notin \mathbb{Q} \end{cases}$; For $x \in \mathbb{R} \setminus \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; For $x \in \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M|g(x)|$; □

Binary relation

$\Theta(g(x))$ is an equivalence class.

Proof. Transitive: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$; $\exists N_1, N_2 > 0, U \in \mathbb{R} : x \geq U \implies N_1|h(x)| \leq |g(x)| \leq N_2|h(x)|$; then $\exists M_1N_1, M_2N_2 > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies M_1N_1|h(x)| \leq |f(x)| \leq M_2N_2|h(x)|$; Reflexive: $f(x) \in \Theta(f(x))$; $\exists 1, 1 > 0, a \in \mathbb{R} : x \in \mathbb{R} \implies 1|f(x)| \leq |f(x)| \leq 1|f(x)|$; Symmetric: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1|g(x)| \leq |f(x)| \leq M_2|g(x)|$; then $\exists \frac{1}{M_2}, \frac{1}{M_1} > 0, T \in \mathbb{R} : x \geq T \implies \frac{1}{M_2}|f(x)| \leq |f(x)| \leq \frac{1}{M_1}|f(x)|$; □

$O(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Transitive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N|h(x)|$; then $\exists MN > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| \leq MN|h(x)|$; Reflexive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M}|f(x)|$; Antisymmetric: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N|f(x)|$; then $\exists M, \frac{1}{N} > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies \frac{1}{N}|g(x)| \leq |f(x)| \leq M|g(x)|$; □

$\Omega(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Similar to that of O. □

$o(g(x))$ is a strictly partially-ordered set.

Proof. Asymmetry: $f(x) \in o(g(x)) \iff g(x) \in \omega(f(x)) \implies g \notin o(f(x))$; Transitivity: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M|g(x)|$; $\exists U \in \mathbb{R} : x \geq U \implies |g(x)| < |h(x)|$; then $\forall M > 0 : \exists \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| < M|h(x)|$; □

$\omega(g(x))$ is a strictly partially-ordered set.

Proof. Similar to that of o. □

Integral test (for convergence) or Maclaurin – Cauchy test

Suppose $f(x) : [a, \infty) \to \mathbb{R}$ for some $a \in \mathbb{N}$ is monotone and $\lim_{x\to\infty} f(x) = 0$. Then $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent iff $\sum_{k=a}^{\infty} f(k)$ is convergent. And,

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone decreasing, for any $n \in \mathbb{N}_{\geq a}$, $\int_n^\infty f(x)\,\mathrm{d}x \leq \sum_{k=n}^{\infty} f(k) \leq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \leq \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone increasing, for any $n \in \mathbb{N}_{\geq a}$, $\int_n^\infty f(x)\,\mathrm{d}x \geq \sum_{k=n}^{\infty} f(k) \geq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \geq \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

Proof. Consider the case where $f(x)$ is monotone increasing. The case where $f(x)$ is monotone decreasing can be proven similarly. First consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent. This implies that $f(x)$ is non-positive. Thus, for any fixed $n \in \mathbb{N}_{\geq a}$, $\int_n^{n+1} f(x)\,\mathrm{d}x \geq f(n)$.; and for any $k \geq n+1$, $\int_k^{k+1} f(x)\,\mathrm{d}x \geq f(k) \geq \int_{k-1}^{k} f(x)\,\mathrm{d}x$.; For any $N \in \mathbb{N}_{\geq n}$, summing over all $k \in \mathbb{N}_{\geq n}$, we obtain, $\int_n^{N+1} f(x)\,\mathrm{d}x \geq \sum_{k=n}^{N} f(k) \geq f(n) + \int_n^N f(x)\,\mathrm{d}x$.; Take limit $\lim_{N\to\infty}$, we obtain $\int_n^\infty f(x)\,\mathrm{d}x \geq \sum_{k=n}^{\infty} f(n) \geq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; Second, consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is divergent. Since $f(x)$ is monotone. Either $\int_a^\infty f(x)\,\mathrm{d}x = -\infty$ or $\int_a^\infty f(x)\,\mathrm{d}x = \infty$. For the former, the supremum of $f(x)$ is non-positive, and $\sum_{k=a}^{\infty} f(k) = -\infty$. For the latter, there exists $x_0 > a$ such that $f(x) > 0$ for all $x \geq x_0$, and thus $\sum_{k=a}^{\infty} f(k) = \infty$. □

p-test (for improper integral)

For any fixed $a > 0$, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p > 1$, and if so, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$.; $\int_0^a \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p < 1$, and if so, $\int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$.;

Proof. $\int \frac{1}{x^p}\,\mathrm{d}x = \begin{cases} -\frac{1}{(p-1)\,x^{p-1}}, & p \neq 1 \\ \ln(x), & p = 1 \end{cases}$;

$\forall p < 1 : \nexists \lim_{x\to\infty} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x\to\infty} \ln(x)$; $\forall p > 1 : \lim_{x\to\infty} -\frac{1}{(p-1)\,x^{p-1}} = 0 \quad \int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$; $\forall p > 1 : \nexists \lim_{x\to 0^+} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x\to 0^+} \ln(x)$; $\forall p < 1 : \lim_{x\to 0^+} -\frac{1}{(p-1)\,x^{p-1}} = 0 \quad \int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$; □

Generalized p-test (for improper integral)

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x$, $x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right)$, $n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a > 1, k \in \mathbb{N}$, $\int_a^\infty \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)\left(L_k(x)\right)^m}\,\mathrm{d}x$; converges iff $m > 1$.

Proof. Let $u = L_k(x)$; $\mathrm{d}u = \frac{\mathrm{d}x}{\prod_{i=0}^{k-1} L_i(x)}$; $\int \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)\left(L_k(x)\right)^m}\,\mathrm{d}x = \int \frac{1}{u^m}\,\mathrm{d}u$; By p-test, the statement holds. □

For any fixed $0 < a < 1$, $\int_0^a \frac{1}{x(\ln x)^m}\,\mathrm{d}x$; converges iff $m < 1$.

Proof. By substitution of $u = \ln x$. □

Generalized p-test for series or generalized p-series test

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x$, $x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right)$, $n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a \in \mathbb{N}$, $\sum_{n=a}^{\infty} \frac{1}{\left(\prod_{i=0}^{k-1} L_i(n)\right)\left(L_k(n)\right)^m}$; converges iff $m > 1$.

Proof. By integral test and generalized p-test for improper integral, the statement holds. □

Generalized Master Theorem

For a positive valued net $T(n)$ defined on $n = b^h$ for any $h \in \mathbb{N}_0$ as $T(n) = aT\left(\frac{n}{b}\right) + f(n)$; for some constants $a > 0$ and $b > 1$, and non-negative-valued function $f(n)$,

If for some $c < \log_b a$, $f(n) = O\left(n^c\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x$, $x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right)$, $n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.;

If for some positive integers k and positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^{m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\left(L_k(n)\right)^m\right)$.;

If for some positive integer k, $f(n) = \Theta\left(n^{\log_b a}\prod_{i=1}^{k}\left(L_i(n)\right)^{-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}L_{k-1}(n)\right)$.;

If for some positive integers k and some positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^{-m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

If $f(n) = \Omega\left(n^c\right)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, k < 1$ such that $n \geq m$ implies $af\left(\frac{n}{b}\right) \leq kf(n)$; is satisfied, then $T(n) = \Theta(f(n))$.;

Proof. Let $s = \log_b a$, $n = b^h$, and $f(x) = x^s g(x)$. At level j, there are a^j nodes, each of size $\frac{n}{b^j}$. Thus, the total non-recursive work at level j is $a^j f\left(\frac{n}{b^j}\right)$.; Hence $T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) = \sum_{j=0}^{h-1} b^{sj} f\left(\frac{b^h}{b^j}\right) + \Theta\left(n^s\right) = n^s \sum_{j=0}^{h-1} g\left(b^{h-j}\right) + \Theta\left(n^s\right) = n^s \sum_{r=1}^{h} g\left(b^r\right) + \Theta\left(n^s\right)$; Case 1: If for some $c < \log_b a$, $f(n) = O\left(n^c\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 2: $L_k\left(b^x\right) = \Theta\left(L_{k-1}(x)\right)$; $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^{m-1}\right)$; $g\left(b^r\right) = \Theta\left(\left(\prod_{i=1}^{k-2}\left(L_i(r)\right)^{-1}\right)\left(L_{k-1}(r)\right)^{m-1}\right)$; By generalized p-test for series: Sub-case 1 $m > 0$: $T(n) = \Theta\left(n^{\log_b a}\left(L_k(n)\right)^m\right)$.; Sub-case 2 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}L_{k-1}(n)\right)$.; Sub-case 3 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 3: If $f(n) = \Omega\left(n^c\right)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, 0 < k < 1$ such that $n \geq m$ implies $af\left(\frac{n}{b}\right) \leq kf(n)$; is satisfied, then $af\left(\frac{n}{b}\right) \leq kf(n)$; $a^j f\left(\frac{n}{b^j}\right) \leq a^{j-1}kf\left(\frac{n}{b^{j-1}}\right) \leq k^j f(n)$; $T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) = f(n) + \sum_{j=1}^{h-1} O\left(k^j f(n)\right) + \Theta\left(n^s\right)$; $\sum_{j=1}^{\infty} k^j f(n) = \frac{f(n)}{1-k}$; $T(n) = f(n) + O(f(n)) + \Theta\left(n^s\right) = \Theta(f(n))$; □

Terms:

h is called recursion level.

$\log_b a$ is called critical exponent c_{crit}.

$T(1)$ is called leaf cost.

At each node, the first term is called recursive work, and the second term is called non-recursive work.

For original master theorem, the second case is

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}(\ln n)^{m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}(\ln n)^m\right)$.;

If $f(n) = o\left(n^{\log_b a}(\ln n)^{-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\ln(\ln n)\right)$.;

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}(\ln n)^{-m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Stirling's approximation

$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + O\left(\frac{1}{n}\right)\right)$;

Python Operator Precedence Order

`()` > `**` > `+x` `-x` `~x` (bitwise NOT) > `*` `/` `//` `%` > `+` `-` > `«` `»` (preserve sign) > `&` (bitwise AND) > `^` (bitwise XOR) > `|` (bitwise OR) > `==` `!=` `>` `>=` `<` `<=` (value comparison) > `is` `is not` (identity) > `in` `not in` > `not` > `and` > `or`

```
bool(0)  # False, only 0 False, other numbers
         True
2**72  # int is of infinite precision, ** is
       power
pow(2, 72)  # same as 2**72
float("inf")  # infinite float
(1).__add__(2)  # 3, must use (1). since 1.
       is treated as decimal
3 // 2  # 1, floor quotient
3 / 2  # 1.5, quotient
divmod(3, 2)  # (1, 1), the pair (x // y, x %
       y)
abs(-2)  # 2
c = complex(1, 2)  # (1+2j), complex(re, im)
c.conjugate()  # (1-2j)
a = 0.5  # a == 0.5
type(True)  # <class 'bool'>
type(1)  # <class 'int'>
type(1.0)  # <class 'float'>
type(complex(1, 1))  # <class 'complex'>


a = "a"
type(a)  # <class 'str'>
a += "b"  # a == 'ab'
"He".__add__("llo")  # 'Hello'
str(1)  # '1'
"g" in "egg"  # True
"f" in "egg"  # False
len("Hello")  # 5
"Hello World".capitalize()  # 'Hello world',
       first capitalize, other lowercase
# str.center(width, fillchar=' ') # Return
       centered in a string of length width
       . Padding is done using the
       specified fillchar (default is an
       ASCII space). The original string is
       returned if width is less than or
       equal to len(s).
"Python".center(10)  #  '  Python  '
"Python".center(10, "-")  # '--Python--'
"Python".center(4)  # 'Python'
# str.count(sub[, start[, end]])
"spam, spam, spam".count("spam")  # 3
"spam, spam, spam".count("spam", 5)  # 2
"spam, spam, spam".count("spam", 5, 10)  # 1
"spam, spam, spam".count("eggs")  # 0
"spam, spam, spam".count("")  # 17
# str.endswith(suffix[, start[, end]]),
       Return True if the string ends with
       the specified suffix, otherwise
       return False. suffix can also be a
       tuple of suffixes to look for. With
       optional start, test beginning at
       that position. With optional end,
       stop comparing at that position.
       Using start and end is equivalent to
       str[start:end].endswith(suffix).
# suffix can also be a tuple of suffixes to
       look for.
"Python".endswith("on")  # True
"a tuple of suffixes".endswith(("at", "in"))
       # False
"a tuple of suffixes".endswith(("at", "es"))
       # True
"Python is amazing".endswith("is", 0, 9)  #
       True
# str.startswith(prefix[, start[, end]]),
       endswith but startswith
# str.expandtabs(tabsize=8)
"01\t012\t0123\t01234".expandtabs()  # '01
       012     0123    01234'
"01\t012\t0123\t01234".expandtabs(4)  # '01
       012 0123    01234'
# str.find(sub[, start[, end]]), Return the
       lowest index in the string where
       substring sub is found within the
       slice s[start:end]. Optional
       arguments start and end are
       interpreted as in slice notation.
       Return -1 if sub is not found.
"spam, spam, spam".find("sp")  # 0
"spam, spam, spam".find("sp", 5)  # 6
# str.rfind(sub[, start[, end]]), find but
       highest index
# str.index(sub[, start[, end]]), Like find
       (), but raise ValueError when the
       substring is not found.
"spam, spam, spam".index("spam")  # 0
# 'spam, spam, spam'.index('eggs') #
       ValueError: substring not found
# str.rindex(sub[, start[, end]]), index but
       highest index
# str.join(iterable), Return a string which
       is the concatenation of the strings
       in iterable. A TypeError will be
       raised if there are any non-string
       values in iterable, including bytes
       objects. The separator between
       elements is the string providing
       this method.
", ".join(["spam", "spam", "spam"])  # 'spam,
       spam, spam'
"-".join("Python")  # 'P-y-t-h-o-n'
"".join(["a", "b", "c"])  # 'abc'
"Hello World".lower()  # 'hello world', all
       lowercase
# str.lstrip(chars=None), Return a copy of
       the string with leading characters
       removed. The chars argument is a
       string specifying the set of
       characters to be removed. If omitted
       or None, the chars argument
       defaults to removing whitespace. The
       chars argument is not a prefix;
       rather, all combinations of its
       values are stripped.
"   spacious   ".lstrip()  # 'spacious   '
"www.example.com".lstrip("cmowz.")  # '
       example.com'
# str.rstrip(chars=None), lstrip but trailing
# str.strip(chars=None), lstrip and rstrip
# str.partition(sep), Split the string at the
       first occurrence of sep, and return
       a 3-tuple containing the part
       before the separator, the separator
       itself, and the part after the
       separator. If the separator is not
       found, return a 3-tuple containing
       the string itself, followed by two
       empty strings.
"Monty Python".partition(" ")  # ('Monty', '
       ', 'Python')
"Monty Python's Flying Circus".partition(
    " ")
)  # ('Monty', ' ', "Python's Flying Circus")
"Monty Python".partition("-")  # ('Monty
       Python', '', '')
# str.rpartition(sep), partition but last
       occurrence
# str.removeprefix(prefix), If the string
       starts with the prefix string,
       return string[len(prefix):].
       Otherwise, return a copy of the
       original string:
"Arthur: three!".removeprefix("Arthur: ")  #
       'three!'
# str.removesuffix(suffix, /), If the string
       ends with the suffix string and that
       suffix is not empty, return string
       [:-len(suffix)]. Otherwise, return a
       copy of the original string
# str.replace(old, new, count=-1), Return a
       copy of the string with all
       occurrences of substring old
       replaced by new. If count is given,
       only the first count occurrences are
       replaced. If count is not specified
       or -1, then all occurrences are
       replaced.
"spam, spam, spam".replace("spam", "eggs")  #
       'eggs, eggs, eggs'
"spam, spam, spam".replace("spam", "eggs", 1)
       # 'eggs, spam, spam'
# str.split(sep=None, maxsplit=-1), Return a
       list of the words in the string,
       using sep as the delimiter string.
       If maxsplit is given, at most
       maxsplit splits are done, the
       leftmost ones (thus, the list will
       have at most maxsplit+1 elements).
       If maxsplit is not specified or -1,
       then there is no limit on the number
       of splits (all possible splits are
       made). If sep is given, consecutive
       delimiters are not grouped together
       and are deemed to delimit empty
       strings (for example, '1,,2'.split
       (',') returns ['1', '', '2']). The
       sep argument may consist of multiple
       characters as a single delimiter.
       Splitting an empty string with a
       specified separator returns ['']. If
       sep is not specified or is None, a
       different splitting algorithm is
       applied: runs of consecutive
       whitespace are regarded as a single
       separator, and the result will
       contain no empty strings at the
       start or end if the string has
       leading or trailing whitespace.
       Consequently, splitting an empty
       string or a string consisting of
       just whitespace with a None
       separator returns [].
"1,2,3".split(",")  # ['1', '2', '3']
"1,2,3".split(",", maxsplit=1)  # ['1',
       '2,3']
"1,2,,3,".split(",")  # ['1', '2', '', '3',
       '']
"1<>2<>3<4".split("<>")  # ['1', '2', '3<4']
# str.rsplit(sep=None, maxsplit=-1), split
       but rightmost
# str.swapcase()
"Hello World".swapcase()  # 'hELLO WoRLD'
# str.title(), Return a titlecased version of
       the string where words start with
       an uppercase character and the
       remaining characters are lowercase.
       The algorithm uses a simple language
       -independent definition of a word as
       groups of consecutive letters.
"Hello world".title()  # 'Hello World'
"they're bill's friends from the UK".title()
       # "They'Re Bill'S Friends From The
       Uk"
# str.upper(), all uppercase


a = [10, 20, 30]  # list is a dynamic array
       of elements of arbitrary types
type(a)  # <class 'list'>
a[0]  # 10, O(1)
a[1]  # 20, O(1)
a[-1]  # 30, O(1)
a[-2]  # 20, O(1)
len(a)  # 3, O(1)
a.__len__()  # 3, O(1)
a.append(40)  # a == [10, 20, 30, 40],
       amortized O(1)
a.pop()  # a == [10, 20, 30], O(1)
a.insert(1, 50)  # a == [10, 50, 20, 30], O(n
       )
a.pop(0)  # a == [50, 20, 30], O(n)
a[0] = 0  # a == [0, 20, 30], O(1)
b, c, d = a  # b == 0, c == 20, d == 30
a.clear()  # a == []
a = [0, 1, 2]
b = a  # id(a) == id(b)
b.pop()  # a == b == [0, 1], id(a) == id(b)
b = [1, 2]  # id(a) != id(b), a == [0, 1]
a = [1, 1, 1]
a.count(1)  # 3, O(n)
b = a.copy()  # a == b but id(a) != id(b)
a.index(1)  # 0, first occurrence, O(n)
a.extend([0, 4, 3, 4]),  # a == [1, 1, 1, 0,
       4, 3, 4], O(k)
a.remove(4)  # a.pop(a.index(x))
a.reverse()  # a == [4, 3, 0, 1, 1, 1], O(n)
a.sort()  # [0, 1, 1, 1, 3, 4], O(n log n)
       worst-case O(n) best-case, Powersort
sorted(a)  # same as a.sort()
del a[0:5:2]  # a == [1, 1, 4]
1 in a  # True
0 not in a  # True
a = [None] * 5  # a == [None, None, None,
       None, None]
del a  # a is now not defined
a = [1, "1", [1], True]  # arbitrary types
it = [1, 2, 3].__iter__()
next(it)  # 1, also it.__next__()
# Return of __next__ is defined by the yield
       clause in the method.
# __next__ is used by for loop.
# range(start, stop, step), all are integers,
       start + k step, k = [0, (stop -
       start) / step - 1] integers
list(range(0, 10, 1))  # [0, 1, 2, 3, 4, 5,
       6, 7, 8, 9]
list(range(9, -1, -2))  # [9, 7, 5, 3, 1]
# List comprehensions provide a concise way
       to create lists. Common applications
       are to make new lists where each
       element is the result of some
       operations applied to each member of
       another sequence or iterable, or to
       create a subsequence of those
       elements that satisfy a certain
       condition.
list(map(lambda x: x**2, range(10)))  # [0,
       1, 4, 9, 16, 25, 36, 49, 64, 81]
[x**2 for x in range(10)]  # [0, 1, 4, 9, 16,
       25, 36, 49, 64, 81]
[
    (x, y) for x in [1, 2, 3] for y in [3, 1,
       4] if x != y
]  # [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4),
       (3, 1), (3, 4)]
[
    w.strip() for w in ["  banana", "
       loganberry ", "passion fruit  "]
]  # ['banana', 'loganberry', 'passion fruit
       ']
[
    num for elem in [[1, 2, 3], [4, 5, 6],
       [7, 8, 9]] for num in elem
]  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
[
    [
        row[i]
        for row in [
            [1, 2, 3, 4],
            [5, 6, 7, 8],
            [9, 10, 11, 12],
        ]
    ]
    for i in range(4)
]  # [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4,
       8, 12]]
# zip(*iterables, strict=False), Iterate over
       several iterables in parallel,
       producing tuples with an item from
       each one, similar to transposing a
       matrix. By default, zip() stops when
       the shortest iterable is exhausted.
       With strict=True, it raises a
       ValueError if one iterable is
       exhausted before the others.
list(
    zip(
        *[
            [1, 2, 3, 4],
            [5, 6, 7, 8],
            [9, 10, 11, 12],
        ]
    )
)  # [(1, 5, 9), (2, 6, 10), (3, 7, 11), (4,
       8, 12)]
list(zip(range(3), ["fee", "fi", "fo", "fum"
       ]))  # [(0, 'fee'), (1, 'fi'), (2, '
       fo')]
# list(zip(range(3), ['fee', 'fi', 'fo', 'fum
       '], strict=True)) # ValueError: zip
       () argument 2 is longer than
       argument 1
[0, *[1, 2, 3], 4, 5, 6]  # [0, 1, 2, 3, 4,
       5, 6]


a = (
    1,
    "a",
    "a",
    [0],
)  # tuple is an immutable ordered collection
       of arbitrary number of elements of
       arbitrary types
type(a)  # <class 'tuple'>
a = 1, "a", "a", [0]  # can also be created
       without the parentheses
a[0]  # 1
a[-1]  # [0]
# a[0] = 0 # TypeError: 'tuple' object does
       not support item assignment
a[3][0] = 1  # allowed
a = ("a",)  # a == ('a',), a tuple. One-item
       tuple requires comma
a = ("a",)  # a == ('a',), a tuple
a = "a"  # a == 'a', a string
a = (1, 1, 1)
a.count(1)  # 3
a.index(1)  # 0, first occurrence
1 in a  # True
0 not in a  # True


a = {
    1,
    "a",
    "a",
}  # t == {1, 'a'}, set is an immutable
       unordered collection of arbitrary
       number of elements of hashable types
       with duplicates ignored
"a".__hash__()  # -8760825294024040742, a
       hashable type has __hash__ method
       that returns an int
# a = {[0]} # TypeError: cannot use 'list' as
       a set element (unhashable type: '
       list')
type(a)  # <class 'set'>
type({})  # <class 'dict'>
a = {True, 1}  # t == {True}
a = {1, True}  # t == {1}
a = {False, 0}  # t == {False}
a = {0, False}  # t == {0}
len(a)  # 1
a.clear()  # a == set()
a = {1, 0, 2}  # a == {0, 1, 2}

a.add(3)  # a == {0, 1, 2, 3}
a.update([1])  # a == {0, 1, 2, 3}, take any
       iterable and add all its elements
a.update([1])  # a == {0, 1, 2, 3}
a.update((1,))  # a == {0, 1, 2, 3}
b = a.copy()  # id(a) != id(b)
a.difference({1})  # {0, 2, 3}, a.difference(
       b) returns a \setminus b, take any
       iterable
a.difference({1})  # {0, 2, 3}
a.difference_update(
    {1}
)  # a == {0, 2, 3}, same as difference but
       not return but assign to a


# 'difference', 'difference_update', 'discard
       ', 'intersection', '
       intersection_update', 'isdisjoint',
       'issubset', 'issuperset', 'pop', '
       remove', 'symmetric_difference', '
       symmetric_difference_update', 'union
       ', 'update'
{c for c in "abracadabra" if c not in "abc"}
       # {'d', 'r'}, set comprehension


class ListStack(list):  # LIFO O(1)
    def push(self, x):
        return self.append(x)

    def top(self):
        return self[-1]

    def is_empty(self):
        return self.__len__() == 0


class _SNode:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next


class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
        return self.head is None

    def prepend(self, x):
        self.head = _SNode(x, self.head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def append(self, x):
        self.tail = _DNode(x, None)
        if self.head is None:
            self.head = self.tail
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("remove from
                empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek at empty
                list")
        return self.head.data

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek at empty
                list")
        return self.tail.data

    def clear(self):
        for _ in range(self.size):
            self.pop_front()

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next


class LinkedStack(SinglyLinkedList):  # LIFO
       O(1)
    def push(self, x):
        return self.prepend(x)

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()


class LinkedQueue(SinglyLinkedList):  # FIFO
       O(1)
    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def peek(self):
        return self.peek_front()


class _DNode:
    def __init__(self, data, prev=None, next=
           None):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
        return self.size == 0

    def prepend(self, x):
        self.head = _DNode(x, None, self.head
            )
        if self.tail is None:
            self.tail = self.head
        self.head.next.prev = self.head
        self.size += 1

    def append(self, x):
        self.tail = _DNode(x, self.tail, None
            )
        if self.head is None:
            self.head = self.tail
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("pop from empty
                list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        else:
            self.head.prev = None
        return x

    def pop_back(self):
        if self.tail is None:
            raise IndexError("pop from empty
                list")
        x = self.tail.data
        self.tail = self.tail.prev
        self.size -= 1
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek at empty
                list")
        return self.head.data

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek at empty
                list")
        return self.tail.data

    def clear(self):
        for _ in range(self.size):
            self.pop_back()

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next


class LinkedDequeue(DoublyLinkedList):  #
       Both sides IO O(1)
    def push(self, x):
        return self.prepend(x)

    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()

    def peek(self):
        return self.peek_front()


class BlockListDeque:  # Both sides IO
       amortized O(1)
    def __init__(self, block_size=64,
           front_step=1, back_step=1):
        assert block_size > 0
        assert front_step > 0
        assert back_step > 0
        self.block_size = block_size
        self.front_step = front_step
        self.back_step = back_step
        self.data = [
            [None] * self.block_size
            for _ in range(self.front_step +
                self.back_step - 1)
        ]
        self.block_begin = 0
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1)
            // 2
        self.in_end = self.in_begin
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
        return self.size == 0

    def capacity(self):
        return self.block_size * len(self.
            data)

    def is_valid(self):
        return (
            self.size
            == self.block_size * (self.
                block_end - self.
                block_begin - 1)
            + self.in_end
            - self.in_begin
        )

    def reserve_front(self, blocks=None):
        if blocks is None:
            blocks = self.front_step
        new = [[None] * self.block_size for _
            in range(blocks)]
        new.extend(self.data)
        self.data = new
        self.block_begin += blocks
        self.block_end += blocks

    def reserve_back(self, blocks=None):
        if blocks is None:
            blocks = self.back_step
        self.data.extend([[None] * self.
            block_size for _ in range(
            blocks)])

    def shrink_front_to_fit(self):
        new = self.data[self.block_begin :]
        self.data = new
        self.block_end -= self.block_begin
        self.block_begin = 0

    def shrink_back_to_fit(self):
        for i in range(self.block_end, len(
            self.data)):
            self.data.pop()

    def prepend(self, x):
        if self.in_begin != 0:
            self.in_begin -= 1
        else:
            if self.block_begin == 0:
                self.reserve_front()
            self.block_begin -= 1
            self.in_begin = self.block_size -
                1
        self.data[self.block_begin][self.
            in_begin] = x
        self.size += 1

    def append(self, x):
        if self.in_end != self.block_size:
            self.in_end += 1
        else:
            if self.block_end == len(self.
                data):
                self.reserve_back()
            self.block_end += 1
            self.in_end = 1
        self.data[self.block_end - 1][self.
            in_end - 1] = x
        self.size += 1

    def pop_front(self):
        if self.is_empty():
            raise IndexError("pop from empty
                deque")
        x = self.data[self.block_begin][self.
            in_begin]
        self.data[self.block_begin][self.
            in_begin] = None
        self.size -= 1
        if self.in_begin != self.block_size -
            1:
            self.in_begin += 1
        else:
            self.block_begin += 1
            self.in_begin = 0
        return x

    def pop_back(self):
        if self.is_empty():
            raise IndexError("pop from empty
                deque")
        x = self.data[self.block_end - 1][
            self.in_end - 1]
        self.data[self.block_end - 1][self.
            in_end - 1] = None
        self.size -= 1
        if self.in_end != 1:
            self.in_end -= 1
        else:
            self.block_end -= 1
            self.in_end = self.block_size
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek from empty
                deque")
        return self.data[self.block_begin][
            self.in_begin]

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek from empty
                deque")
        return self.data[self.block_end - 1][
            self.in_end - 1]

    def clear(self):
        for _ in range(self.size):
            self.pop_back()

    def __iter__(self):
        block = self.block_begin
        index = self.in_begin
        for _ in range(self.size):
            yield self.data[block][index]
            index += 1
            if index == self.block_size:
                index = 0
                block += 1

    def push(self, x):
        return self.prepend(x)

    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()

    def peek(self):
        return self.peek_front()


class BlockListQueue(BlockListDeque):
    pass
```